

OpenEden Vault V5 — Incremental Security Review Report

Contract: OpenEdenVaultV5Impl.sol
Baseline: OpenEdenVaultV4Impl.sol (previously audited)
Date: Aug 7, 2025
Solidity Version: 0.8.9
Review Scope: Delta changes from V4 to V5 only

1. Executive Summary

OpenEdenVaultV5 is an incremental upgrade from the previously audited V4 implementation. The core deposit, redeem, withdrawal queue, fee calculation, and epoch management logic remains **unchanged**. V5 introduces a pluggable redemption system, supply cap controls, administrative mint/burn/reissue capabilities, a dedicated management fee treasury, and several security hardening improvements.

12 findings were identified across the V5 delta. The final distribution is:

Severity	Count
Low	8
Informational	4

All privileged administrative functions (`mintTo`, `burnFrom`, `reIssue`) are gated behind the `maintainer` role, which is operated as a Gnosis Safe multisig. The `owner` role (UUPS upgrade authority) is separately controlled. No critical or high-severity findings remain.

2. Summary of Changes (V4 to V5)

#	Change	Category
1	Hardcoded BUIDL redemption replaced with pluggable <code>IRedemption</code> interface	Architecture
2	New <code>totalSupplyCap</code> with enforcement on deposit and <code>mintTo</code>	Feature
3	New <code>burnFrom()</code> — maintainer can burn tokens from any wallet	Feature
4	New <code>mintTo()</code> — maintainer can mint tokens to any wallet	Feature
5	New <code>reIssue()</code> — maintainer can transfer shares between wallets	Feature
6	Separate <code>mgtFeeTreasury</code> for management fees (previously shared with <code>oplTreasury</code>)	Architecture
7	<code>_beforeTokenTransfer</code> now skips KYC for vault-originated transfers	Behavior
8	Added <code>constructor() { _disableInitializers(); }</code>	Security
9	Added full <code>initialize()</code> function for fresh deployments	Lifecycle
10	<code>redeemIns</code> now enforces <code>minWithdraw</code> check (missing in V4)	Bugfix
11	<code>_processDeposit</code> reordered: supply cap validated before any token transfers	Logic
12	<code>checkLiquidity()</code> removed from vault, delegated to redemption contract	Architecture
13	<code>setBuidl</code> / <code>setBuidlTreasury</code> removed, replaced by <code>setRedemption</code>	Interface
14	Storage slot previously used for <code>maxDepeg</code> repurposed for <code>totalSupplyCap</code>	Storage

3. Findings

F-01 [INFORMATIONAL] Maintainer Role Has Expanded Privileges

Location: `mintTo` (line 780), `burnFrom` (line 765), `reIssue` (line 804)

The maintainer can mint, burn, and reissue shares without on-chain timelock or multi-signature enforcement at the contract level. The contract validates only that `msg.sender == maintainer`.

Mitigation: The maintainer address is a Gnosis Safe multisig wallet, providing multi-party authorization at the operational layer. The `setMaintainer()` function is restricted to `onlyOwner`, preventing unauthorized changes to this critical role.

Recommendation: Ensure operational documentation specifies the required multisig threshold configuration for the maintainer address.

F-02 [LOW] Instant Redemption Trusts External Return Value Without Balance Verification

Location: `_processWithdrawIns` (line 917)

```
uint256 usdcReceived = redemptionContract.redeem(_assets + 1e6);

if (usdcReceived < _assets)
    revert TBillReceiveUSDCFailed(usdcReceived, _assets);
```

V4 performed a before/after balance check to verify actual USDC receipt. V5 trusts the return value from the redemption contract. The redemption contract itself enforces correctness, but the vault no longer has an independent verification layer.

Additionally, the `+ 1e6` rounding buffer is hardcoded, assuming 6-decimal underlying tokens. Excess redeemed USDC remains in the vault and is handled through operational off-ramping.

Recommendation: Consider re-adding a before/after balance check as defense-in-depth, and deriving the rounding buffer from `underlying.decimals()` rather than hardcoding.

F-03 [LOW] `redeemIns` Computes Fee Before Validating Minimum Withdrawal

Location: `redeemIns` (lines 243-246)

```
(, , uint256 totalFee) = txsFee(ActionType.REDEEM, sender, assets);

(uint256 minWithdraw, ) = feeManager.getMinMaxWithdraw();
if (assets < minWithdraw) revert TBillLessThanMin(assets, minWithdraw);
```

The fee is computed via external calls before the minimum withdrawal check. Users who fail the minimum check pay unnecessary gas for the fee computation. The standard `redeem()` function correctly validates the minimum first.

Note: V5 adding the `minWithdraw` check to `redeemIns` is itself a bugfix — V4 had no minimum enforcement on instant redemptions.

Recommendation: Move the minimum check before the fee computation.

F-04 [INFORMATIONAL] Supply Cap Check Correctly Ordered in `_processDeposit`

Location: `_processDeposit` (lines 845-850)

The supply cap is validated before any token transfers occur. This is a correct fail-fast pattern that improves upon V4's ordering. No action needed.

F-05 [INFORMATIONAL] KYC Bypass for Vault-Originated Transfers

Location: `_beforeTokenTransfer` (line 1005)

```
if (_from == address(0) || _to == address(0) || _from == address(this)) return;
```

V5 skips KYC validation when the vault transfers tokens (e.g., returning shares during withdrawal cancellation). This is a deliberate design decision to ensure queued withdrawals can be cancelled even if a user's KYC status changes while shares are held by the vault.

F-06 [LOW] `reIssue` Allows Same Source and Destination Wallet

Location: `reIssue` (lines 804-819)

Calling `reIssue(addr, addr, amount)` would burn then mint to the same address — a no-op that emits misleading events.

Recommendation: Add `if (_oldWallet == _newWallet) revert TBillInvalidInput(0);`

F-07 [LOW] `setOperator` Does Not Validate Array Length Parity

Location: `setOperator` (lines 554-564)

If `_operators` and `statuses` arrays have different lengths, the function will either revert with an out-of-bounds error or silently ignore extra entries. This also exists in V4.

Recommendation: Add `if (_operators.length != statuses.length) revert TBillInvalidInput(_operators.length);`

F-08 [LOW] `initialize()` Does Not Set `totalSupplyCap`

Location: `initialize` (lines 151-176)

After initialization, `totalSupplyCap` is 0, which blocks all deposits until explicitly configured by the maintainer.

Recommendation: Accept `totalSupplyCap` as an initialization parameter, or default to `type(uint256).max`.

F-09 [LOW] `initialize()` Does Not Set `maintainer` or `operators`

Location: `initialize` (lines 151-176)

After initialization, `maintainer` is `address(0)` and no operators are set. The owner must call `setMaintainer()` before the vault becomes operational.

Recommendation: Accept `_maintainer` as an initialization parameter.

F-10 [LOW] No Pre-Flight Check for Unset `redemptionContract`

Location: `_processWithdrawIns` (line 917)

If `redemptionContract` is not yet configured, `redeemIns` will fail with a low-level revert rather than a descriptive error.

Recommendation: Add `if (address(redemptionContract) == address(0)) revert TBillZeroAddress();`

F-11 [INFORMATIONAL] `checkLiquidity()` Removed — Breaking Change for Integrators

V4 exposed `checkLiquidity()` directly on the vault. V5 delegates this to the `IRedemption` interface. External integrators calling `vault.checkLiquidity()` will need to update their integration to query the redemption contract directly.

F-12 [INFORMATIONAL] Constructor `_disableInitializers()` Added

V5 adds `_disableInitializers()` in the constructor, preventing the implementation contract from being initialized directly. This is a security improvement that addresses a known attack vector for UUPS proxies. V4 was missing this protection.

4. Storage Layout Compatibility

The following storage slots have changed semantics between V4 and V5. All slots maintain the same type and position, ensuring binary compatibility. Semantic changes require post-upgrade configuration.

Slot	V4 Variable	V5 Variable	Status
1	reserve1 (maxDepeg)	totalSupplyCap	Repurposed — set post-upgrade
2	reserve2 (maxTimeDelay)	reserve1 (maxTimeDelay)	Compatible
3	tbillDecimalScaleFactor	tbillDecimalScaleFactor	Unchanged
4	underlying	underlying	Unchanged
5	controller	controller	Unchanged
6	maintainer	maintainer	Unchanged
7	tbillUsdPriceFeed	tbillUsdPriceFeed	Unchanged
8	buidlTreasury	reserve2	Repurposed (reserved)
9	oplTreasury	oplTreasury	Unchanged
10	treasury	treasury	Unchanged
11	qTreasury	qTreasury	Unchanged
12	feeManager	feeManager	Unchanged
13	kycManager	kycManager	Unchanged
14	unClaimedFee	unClaimedFee	Unchanged
15	epoch	epoch	Unchanged
16	lastUpdateTS	lastUpdateTS	Unchanged
17	timeBuffer	timeBuffer	Unchanged
18	isWeekend	isWeekend	Unchanged

19	withdrawalQueue	withdrawalQueue	Unchanged
20	firstDepositMap	firstDepositMap	Unchanged
21	reserve5 (depositMap)	reserve3 (depositMap)	Compatible
22	reserve4 (withdrawMap)	reserve4 (withdrawMap)	Unchanged
23	withdrawalInfo	withdrawalInfo	Unchanged
24	partnerShip	partnerShip	Unchanged
25	operators	operators	Unchanged
26	buidlRedemption	redemptionContract	Interface change — set post-upgrade
27	buidl	reserve5	Repurposed (reserved)
28	—	mgtFeeTreasury	New slot — set post-upgrade

Storage layout should be verified with tooling (`hardhat-storage-layout` or `forge inspect`) before deploying any upgrade.

5. Access Control Model

Role	Capabilities	Control
Owner	UUPS upgrade, set treasury addresses, set fee/kyc/controller/price feed contracts, set maintainer	Gnosis Safe multisig
Maintainer	Mint/burn/reissue shares, cancel withdrawals, set operators, set partnership, set redemption contract, set supply cap, set first deposit flags, set time buffer	Gnosis Safe multisig
Operator	Off-ramp funds to treasury, process withdrawal queue, update epoch, set weekend flag, claim service fee	Set by maintainer

V5 expands the maintainer role with `mintTo`, `burnFrom`, `reIssue`, `setRedemption`, and `setTotalSupplyCap`. All privileged operations emit events for off-chain monitoring.

6. Improvements Over V4

- Implementation initialization guard** — `_disableInitializers()` prevents the uninitialized implementation attack vector
- Minimum withdrawal enforcement on instant redeem** — V4 allowed instant redemptions below the configured minimum
- Total supply cap** — Configurable limit prevents unbounded minting
- Fail-fast deposit ordering** — Supply cap validated before token transfers
- Pluggable redemption** — Supports any compliant redemption token, not limited to BUIDL
- Vault transfer KYC bypass** — Ensures withdrawal cancellation works even if user KYC status changes while shares are queued
- Separate management fee treasury** — Cleaner separation between transaction fees and

management fees

8. Full initialization function — Supports fresh proxy deployments

7. Recommendations Summary

Severity	ID	Finding	Recommendation
LOW	F-02	Redemption return value trust	Re-add before/after balance verification
LOW	F-03	Fee computed before min check	Reorder validation in <code>redeemIns</code>
LOW	F-06	reIssue same wallet allowed	Add equality check
LOW	F-07	setOperator array length	Add parity validation
LOW	F-08	initialize missing supplyCap	Add parameter or default
LOW	F-09	initialize missing maintainer	Add parameter
LOW	F-10	No redemption contract check	Add zero-address guard

8. Disclaimer

This report is based on static analysis of the source code delta between OpenEdenVaultV4Impl.sol and OpenEdenVaultV5Impl.sol. It covers only the changes introduced in V5, relying on the previously completed audit of V4 as the security baseline. No dynamic testing, fuzzing, or formal verification was performed. Storage layout compatibility should be independently verified with tooling prior to any production upgrade. This review should complement, not replace, a comprehensive security audit by a professional firm.